COMPUTER NETWORKS LAB

PROJECT REVIEW – 1

NETMIND

Network monitoring with an AI intelligence layer

AI-POWERED NETWORK INTELLIGENCE & LLM NETWORK COPILOT

TEAM MEMBERS

Team Member 1

Mohammed Affan

24BAI0014

Team Member 2

Hardik

24BCI0017

COURSE

Course

Computer Networks Lab

Course Code

BCSE308P

NetMind — AI-Powered Network Intelligence & LLM Network Copilot

Computer Networks Lab (BCSE308P) | Project Review – 1

1. ABSTRACT

Modern computer networks generate large volumes of traffic and performance data, which makes manual monitoring and troubleshooting increasingly difficult. Conventional monitoring tools expose raw information such as packet statistics, latency, throughput, protocol usage and connection details, but leave the interpretation of this information to the user.

This project proposes **NetMind**, an AI-powered network intelligence platform that builds on core Computer Networks concepts and adds Machine Learning (ML) and Large Language Model (LLM) capabilities as an intelligence layer. The system will capture and analyze network traffic, identify protocols such as TCP, UDP, ICMP, DNS and HTTP, and compute key performance parameters including latency, throughput, packet loss, jitter and traffic rate. A network topology and performance module will describe the connected nodes, communication paths and prevailing network conditions, while an ML-based anomaly detection module will analyze traffic features to identify unusual network behavior.

In addition, NetMind will incorporate a Retrieval-Augmented Generation (RAG) pipeline built on a knowledge base of networking concepts, protocols, troubleshooting procedures and project-specific information. An LLM Network Copilot will combine the retrieved knowledge with current network statistics to explain detected problems, answer networking questions and provide human-readable troubleshooting recommendations. The Copilot serves as an advisory interface for human users and does not control the network.

The expected outcome is an integrated network intelligence platform that demonstrates the practical application of TCP/IP, packet analysis, routing, network performance analysis, Machine Learning, embeddings, RAG and LLM technologies.

Keywords: *Network Monitoring, Packet Analysis, TCP/IP, Anomaly Detection, Isolation Forest, Retrieval-Augmented Generation, Large Language Model.*

Table 1: Project and team details

Item	Details
Project Title	NetMind — AI-Powered Network Intelligence & LLM Network Copilot
Course	Computer Networks Lab
Course Code	BCSE308P
Document	Project Review – 1 (Project Proposal)
Team Member 1	Mohammed Affan — 24BAI0014
Team Member 2	Hardik — 24BCI0017

2. PROBLEM STATEMENT

Modern networks continuously generate packets, connections and performance information. Existing monitoring tools can collect and display this information, but interpreting it usually requires considerable technical expertise. A user who observes a high latency value or a sudden increase in connections must still determine what the observation means, which protocol or path is responsible and which corrective action is appropriate.

This project addresses the following challenges:

- High latency
- Packet loss
- Network congestion
- Abnormal traffic
- Excessive connections
- Protocol-related problems
- Routing inefficiencies
- Difficulty interpreting raw network statistics

NetMind aims to combine network monitoring, Machine Learning and LLM-based reasoning so that network analysis becomes more understandable and useful. Measured network data remains the foundation of the system; the ML and LLM components interpret and explain this data rather than replace it.

3. PROJECT OBJECTIVES

The objectives of the project are to:

1. Capture and analyze network traffic.
2. Extract information such as source IP, destination IP, ports, protocols, packet size and timestamps.
3. Analyze TCP, UDP, ICMP, DNS and HTTP traffic.
4. Calculate latency, throughput, packet loss, jitter, traffic rate and bandwidth utilization.
5. Visualize network topology and communication paths.
6. Apply Machine Learning for network anomaly detection.
7. Build a networking knowledge base for RAG.
8. Use embeddings and vector search to retrieve relevant networking information.
9. Develop an LLM-powered Network Copilot.
10. Provide human-readable explanations and troubleshooting recommendations.
11. Demonstrate practical Computer Networks concepts through an integrated implementation.

4. PROPOSED SYSTEM ARCHITECTURE

NetMind is organized as a layered pipeline. Network traffic is captured and analyzed by the core networking modules, and the results are stored as a network state. The AI layer (RAG and LLM) then uses this state to generate explanations that are presented on the dashboard. The key design principle is that real network data always forms the foundation of the system, with the AI components placed on top of the networking components (Figure 1).

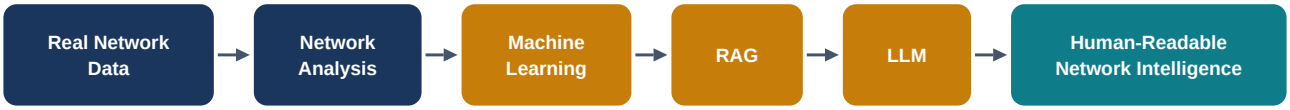

Figure 1: *Design principle of NetMind.*

Design considerations.

- **Networking first:** packet capture, protocol analysis, performance measurement and routing form the core of the system; ML, RAG and the LLM are an intelligence layer built on top of them.
- **Grounded reasoning:** the LLM answers using measured network statistics and retrieved networking knowledge instead of relying on its general training data alone.
- **Advisory role:** the Copilot explains and recommends; it does not control network devices or configuration.
- **Modularity:** each module exposes a clear interface (through FastAPI), so it can be developed and tested independently.
- **Realistic scope:** lightweight components (SQLite, Streamlit) are chosen for the initial version so that the project remains achievable for a student team.

Figure 2 presents the complete proposed architecture and the direction of data flow between the modules.

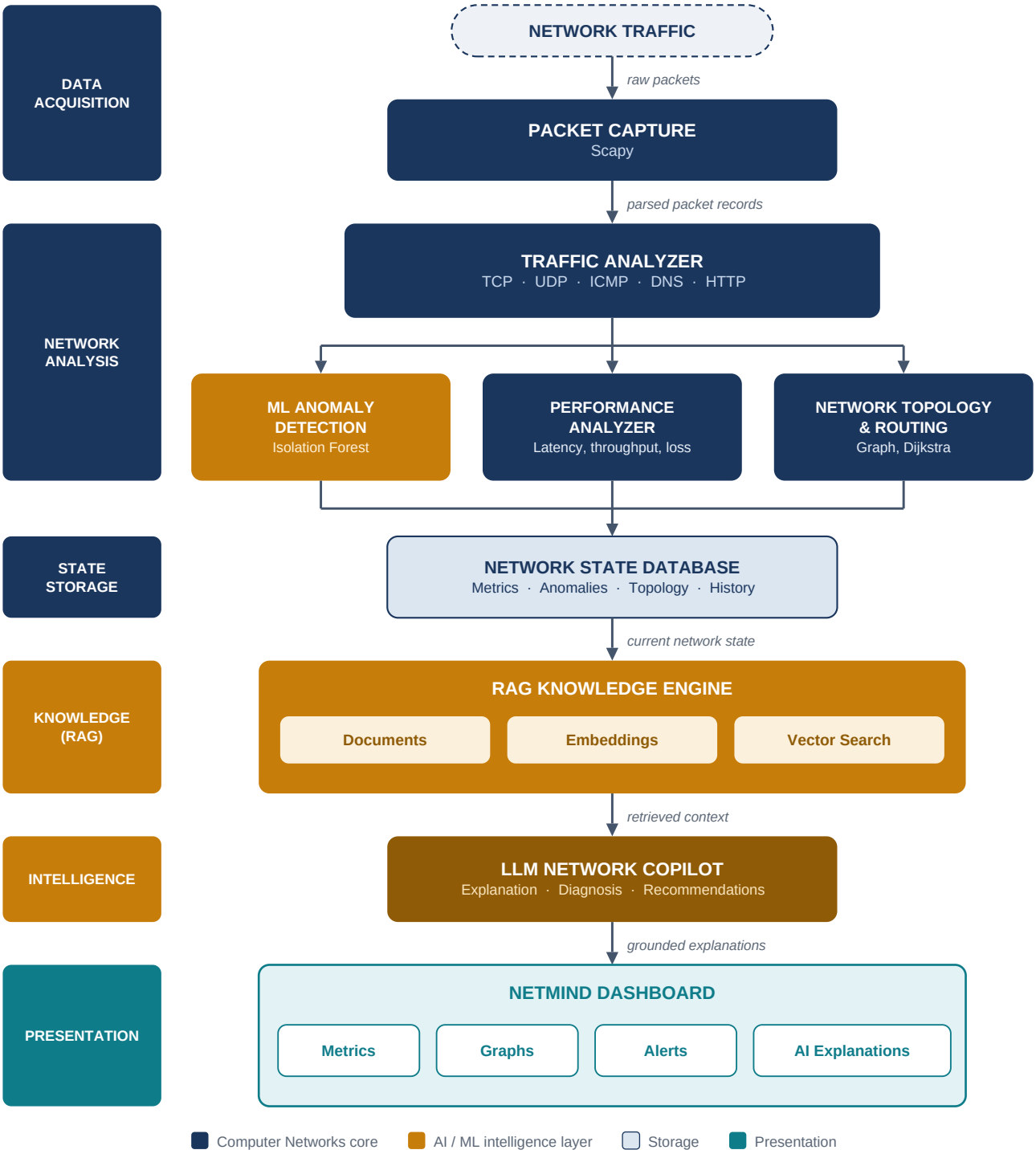

Figure 2: *Proposed system architecture of NetMind.*

5. ARCHITECTURE EXPLANATION

Packet Capture obtains packets from a selected network interface and parses them into structured records. The Traffic Analyzer classifies these records by protocol and groups them into flows. Three modules then operate on the analyzed traffic: ML Anomaly Detection scores traffic behavior, the Performance Analyzer calculates network performance metrics, and the Network Topology and Routing module models nodes, links and paths. Their outputs are written to the Network State Database. When a user asks a question, the RAG Knowledge Engine retrieves relevant networking knowledge, and the LLM Network Copilot combines it with the current network state to produce an explanation that is displayed on the NetMind Dashboard. Table 2 summarizes the responsibility of each layer.

Table 2: Architectural layers and their responsibilities

Layer	Components	Responsibility
Data Acquisition	Packet Capture (Scapy)	Captures packets and extracts header-level fields.
Network Analysis	Traffic Analyzer, Performance Analyzer, Topology and Routing	Classifies protocols, computes performance metrics and models nodes, links and paths.
Machine Learning	ML Anomaly Detection	Scores traffic feature vectors as normal or anomalous.
State Storage	Network State Database	Stores packet, traffic, performance, anomaly and topology records together with history.
Knowledge	RAG Knowledge Engine	Retrieves relevant networking knowledge through semantic search.
Intelligence	LLM Network Copilot	Combines question, network state and retrieved knowledge into grounded explanations.
Presentation	NetMind Dashboard	Displays metrics, graphs, alerts, topology and AI explanations.

6. METHODOLOGY AND MODULE DESCRIPTION

The system is decomposed into nine modules. Each module is described below in terms of its purpose, inputs and outputs.

MODULE 1 — Packet Capture

NetMind will use **Scapy** to capture packets from a selected network interface and convert each packet into a structured record. The following fields will be extracted from every packet: source IP address, destination IP address, source port, destination port, protocol, packet length and timestamp. These records form the raw data layer on which all other modules depend.

MODULE 2 — Traffic Analyzer

The Traffic Analyzer will classify each captured packet as TCP, UDP, ICMP, DNS or HTTP and group packets into flows using addresses, ports and protocol. It will generate traffic statistics such as packet counts, byte counts, active connections and protocol distributions. HTTP analysis is limited to unencrypted traffic, because the content of encrypted sessions cannot be inspected.

MODULE 3 — ML Anomaly Detection

This module converts traffic into feature vectors computed over fixed time windows. The proposed features are packets per second, average packet size, number of connections, connection duration, destination ports, TCP/UDP ratio and traffic rate. The proposed initial model is **Isolation Forest** (Scikit-learn), an unsupervised algorithm that isolates observations that differ from the majority of the data and therefore does not require labeled attack data. For each time window the module will output a classification (normal or anomalous) together with an anomaly score. Model behavior will be evaluated in Phase 4 using traffic generated in a controlled environment.

MODULE 4 — Performance Analyzer

The Performance Analyzer will calculate the metrics summarized in Table 3 from packet timestamps, packet sizes and TCP behavior.

Table 3: Network performance metrics calculated by NetMind

Metric	Definition	Calculation method
Latency	Delay between a request and its response (round-trip time)	RTT = t(response) – t(request)
Throughput	Amount of data delivered per unit time	Mbps = (bytes × 8) / (interval in seconds × 10^6)
Packet loss	Share of transmitted packets that do not reach the destination	Loss (%) = (lost packets / transmitted packets) × 100
Jitter	Variation in delay between successive packets	Mean of |d(i) – d(i–1)| over a window
Traffic rate	Volume of traffic in a time window	Packets per second and bytes per second
Bandwidth utilization	Portion of link capacity in use	Utilization (%) = (throughput / link capacity) × 100

MODULE 5 — Network Topology and Routing

Nodes derived from the endpoints observed in captured traffic will be represented as vertices, and the communication links between them as weighted edges, in a graph built with **NetworkX**. Edge weights may represent measured latency or hop count. **Dijkstra's algorithm** will compute shortest paths between nodes. The routing component can later be extended to Distance Vector and Link State routing simulations.

MODULE 6 — Network State Database

The database will store packet statistics, traffic statistics, performance metrics, anomaly results, network nodes, network connections, protocol information and historical measurements. SQLite is proposed initially because it requires no separate server. Historical records allow the dashboard and the Copilot to compare current behavior with earlier measurements.

MODULE 7 — RAG Knowledge Engine

The RAG Knowledge Engine supplies the Copilot with reliable networking information. Knowledge documents are divided into text chunks, converted into vector embeddings with Sentence Transformers, and stored in a vector database (FAISS or ChromaDB). At query time the question is embedded and the most semantically similar chunks are retrieved (Figure 3).

Figure 3: *Processing stages of the RAG Knowledge Engine.*

The knowledge base is planned to cover the OSI model, TCP/IP model, TCP, UDP, DNS, HTTP, IPv4, IPv6, routing, congestion control and network troubleshooting, together with project-specific information.

MODULE 8 — LLM Network Copilot

The Copilot receives four inputs: the user question, current network statistics, ML anomaly results and retrieved networking knowledge. It produces a network explanation, a diagnosis, a protocol explanation and troubleshooting recommendations in natural language. The Copilot is advisory only; it does not modify network configuration or control network devices.

MODULE 9 — Dashboard

The dashboard will present real-time network metrics, protocol distribution, traffic graphs, the network topology, anomaly alerts, performance statistics and the LLM Network Copilot interface. Streamlit and Plotly are proposed for the initial version (see Section 8).

7. LLM AND RAG ARCHITECTURE

The AI layer of NetMind is based on **Retrieval-Augmented Generation (RAG)**. Figure 4 shows how a user question is combined with retrieved networking knowledge and the current network state before the LLM generates an answer.

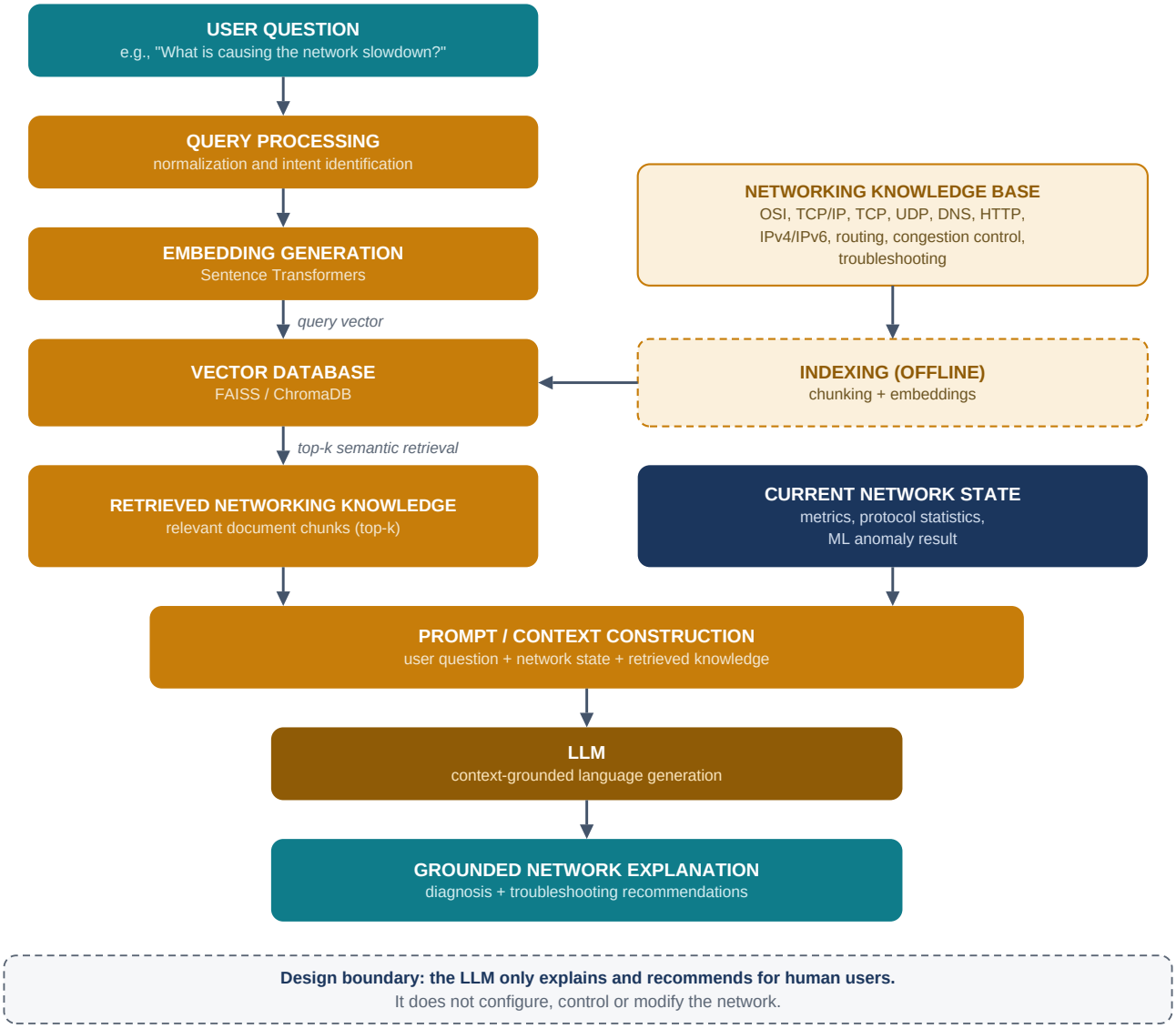

Figure 4: *RAG and LLM pipeline of the NetMind Network Copilot.*

Pipeline. In an offline stage, networking documents are chunked, embedded and indexed in the vector database. At query time the user question is processed and converted into an embedding, and the vector database returns the most relevant document chunks. The system then builds a prompt that combines the retrieved knowledge with the current network state, including the ML anomaly result, and the LLM generates a grounded explanation from this context.

Grounding. Because the prompt contains measured network statistics and retrieved reference material, the LLM is grounded in actual network conditions and established networking knowledge instead of relying only on its general training data. This reduces the risk of unsupported or generic answers.

Safety boundary. The LLM does not independently control the network. It provides explanations and recommendations to human users, who decide whether to act on them.

8. DASHBOARD DESIGN

The dashboard integrates all NetMind outputs in a single interface. Figure 5 shows a conceptual mockup. The values displayed are the illustrative scenario values used in Section 11 and are not measured results.

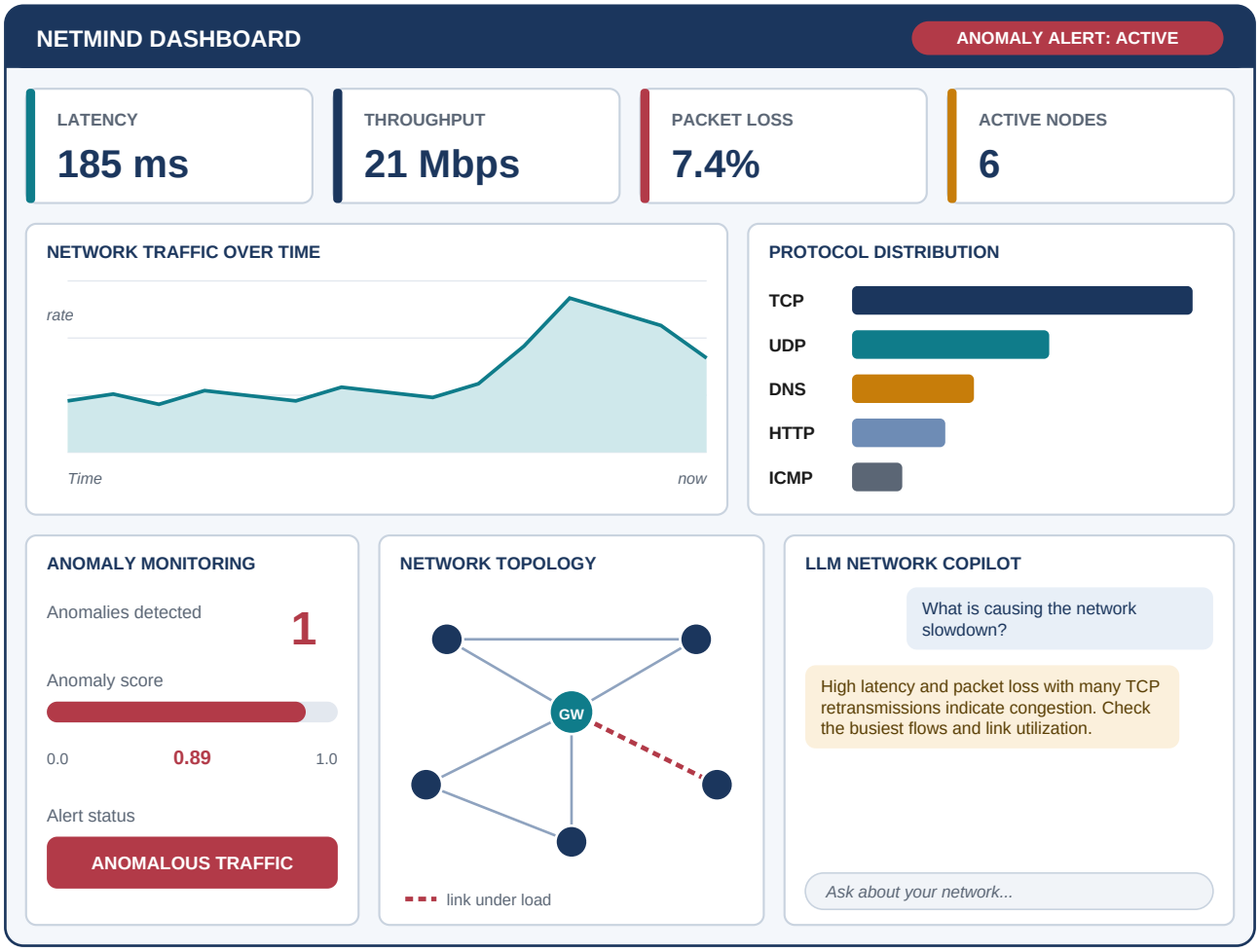

Figure 5: Conceptual mockup of the NetMind Dashboard.

9. TECHNOLOGY STACK

Table 4: Proposed technology stack

Category	Technology	Purpose
Programming	Python	Primary implementation language for all modules.
Packet Capture	Scapy	Captures and dissects packets; extracts header fields.
Network Analysis	NetworkX	Graph modeling of topology; shortest-path computation.
Machine Learning	Scikit-learn	Isolation Forest anomaly detection.
Data Processing	NumPy, Pandas	Feature extraction and time-window statistics.
Backend	FastAPI	REST interfaces for capture, metrics, anomaly and Copilot services.
Frontend	Streamlit (initial); React (possible future alternative)	Dashboard interface; React is a possible professional frontend in later versions.
Visualization	Plotly	Traffic graphs, protocol distribution and performance charts.
Vector Search	FAISS or ChromaDB	Similarity search over knowledge embeddings.
Embeddings	Sentence Transformers	Converts knowledge chunks and queries into vectors.
LLM	LLM API	Generates natural-language explanations from the constructed prompt.
Database	SQLite (initial)	Stores network state and historical measurements.
Version Control	Git + GitHub	Source management and team collaboration.
Testing	Pytest	Unit and integration tests.
Deployment	Docker	Containerized packaging and deployment.

10. COMPUTER NETWORKS SYLLABUS MAPPING

Table 5 maps the project to the course topics. To avoid overstating the scope of the first version, each topic is assigned to one of three categories:

- **Directly implemented** (shown as **Direct** in the table) — the concept is implemented as a working component of NetMind.
- **Captured-traffic analysis** (shown as **Captured traffic**) — the concept is observed and analyzed through captured network traffic; the protocol or mechanism itself is not re-implemented.
- **Planned extension** — the concept is planned as a simulation or extension after the core system is complete.

Table 5: Mapping of NetMind to Computer Networks course topics

No.	Topic	Category	How NetMind demonstrates or relates to the topic
1	Data Communication	**Direct**	Extracts addresses, ports, sizes and timestamps.
2	Network Models	**Captured traffic**	Packets analyzed by layer; OSI covered in the knowledge base.
3	Network Topology	**Direct**	Nodes and links modeled as a graph and visualized.
4	TCP/IP Model	**Captured traffic**	IP, TCP, UDP and ICMP headers parsed from captured packets.
5	Packet Switching	**Captured traffic**	Packet sizes and arrival times observed in captured traffic.
6	Implementing Network Software	**Direct**	Capture, analysis modules and APIs developed in Python.
7	Transmission Impairment	**Captured traffic**	Delay, jitter and loss measured as effects of impairments.
8	Data Rate and Performance	**Direct**	Throughput, traffic rate and bandwidth utilization calculated.
9	Flow Control	**Captured traffic**	TCP window behavior observed in captured segments.

No.	Topic	Category	How NetMind demonstrates or relates to the topic
10	IPv4 Addressing	**Direct**	IPv4 addresses extracted and used to identify nodes.
11	Classless Addressing	**Planned extension**	CIDR-based subnet grouping of nodes in the topology module.
12	NAT	**Planned extension**	NAT behavior to be illustrated through a simulation.
13	IPv6	**Captured traffic**	IPv6 packets parsed where present; full support is future scope.
14	Routing Protocols	**Direct**	Dijkstra shortest-path computation on the topology graph.
15	Link State Routing	**Planned extension**	Link State simulation using shortest-path-first on the graph.
16	Distance Vector Routing	**Planned extension**	Distance Vector simulation using Bellman–Ford.
17	Performance Analysis	**Direct**	Latency, throughput, loss and jitter calculated and stored.
18	TCP	**Captured traffic**	TCP flags, connections and retransmissions analyzed.
19	UDP	**Captured traffic**	UDP flows, datagram sizes and rates analyzed.
20	Congestion Control	**Captured traffic**	Retransmissions, loss and throughput drops as congestion signs.
21	QoS Parameters	**Direct**	Delay, jitter, loss and throughput monitored as QoS metrics.
22	DNS	**Captured traffic**	DNS queries and responses identified and analyzed.
23	HTTP	**Captured traffic**	Unencrypted HTTP requests and responses analyzed.
24	Application Layer	**Captured traffic**	Application protocols classified; results shown on the dashboard.

11. EXAMPLE USE CASE

Scenario. The monitored network suddenly experiences high latency. The network state recorded by NetMind is shown in Table 6. These are illustrative scenario values used to explain the intended behavior of the system; they are not measured results.

Table 6: Network state in the example scenario (illustrative values)

Parameter	Scenario value	Produced by
Latency	185 ms	Performance Analyzer
Packet Loss	7.4%	Performance Analyzer
Throughput	21 Mbps	Performance Analyzer
TCP Retransmissions	High	Traffic Analyzer
Anomaly Score	0.89 (traffic flagged as anomalous)	ML Anomaly Detection

The system processes the scenario as follows:

1. Packet capture and traffic analysis record the traffic and reveal a high number of TCP retransmissions.
2. Performance analysis calculates latency, packet loss and throughput.
3. The ML module flags the traffic window as anomalous.
4. The RAG system retrieves knowledge about TCP retransmissions, packet loss, network congestion and latency.
5. The user asks: *"What is causing the network slowdown?"* The LLM receives the question, the current network state and the retrieved knowledge.
6. The LLM generates a grounded explanation with troubleshooting suggestions.

Figure 6: *Complete pipeline for the example use case.*

Illustrative Copilot output (intended format only; not produced by an implemented system)

The slowdown is most likely caused by network congestion. High latency, packet loss and TCP retransmissions occur together, and the anomaly detector has flagged the current traffic window. When packets are lost, TCP retransmits the missing segments and reduces its sending rate, which increases delay and lowers effective throughput. **Suggested checks:** (1) identify the flows with the highest traffic rate; (2) compare current throughput with link capacity to evaluate bandwidth utilization; (3) inspect the topology view for a congested or faulty link; (4) confirm whether latency and loss return to normal after the load is reduced.

12. EXPECTED OUTCOME

At the end of the project, NetMind is expected to:

1. Capture network traffic.
2. Identify common protocols.
3. Extract network-level information.
4. Monitor network performance.
5. Visualize network topology.
6. Detect anomalous traffic using ML.
7. Store historical network information.
8. Retrieve networking knowledge using RAG.
9. Generate grounded LLM explanations.
10. Provide troubleshooting recommendations.
11. Demonstrate core Computer Networks concepts in a practical system.

13. INNOVATION AND KEY CONTRIBUTION

The main contribution of NetMind is the integration of the following areas into one system:

Figure 7: *Areas integrated by NetMind.*

NetMind does not merely attach an LLM chatbot to a network tool. The Copilot is grounded in actual network statistics collected by the system, ML anomaly results and documents retrieved from a networking knowledge base. This makes each explanation traceable to real observations and established networking concepts.

14. PROJECT DEVELOPMENT PHASES

Table 7: Phased development plan

Phase	Focus	Key tasks
Phase 1	Core networking infrastructure	Packet capture, packet parsing, TCP/UDP analysis
Phase 2	Traffic and performance analysis	Latency, throughput, packet loss, jitter, protocol statistics
Phase 3	Topology and routing	Network graph, nodes, connections, shortest path
Phase 4	Machine Learning	Feature engineering, anomaly detection, model evaluation
Phase 5	RAG	Knowledge base, chunking, embeddings, vector search
Phase 6	LLM Copilot	Context construction, LLM integration, grounded explanations, troubleshooting recommendations
Phase 7	Dashboard and integration	Visualization, alerts, AI interface, testing
Phase 8	Final testing and deployment	Unit tests, integration tests, Docker, documentation, GitHub

15. FUTURE SCOPE

The following items are **FUTURE SCOPE**. They are not part of the planned first implementation and are not claimed as current features.

- Real-time network threat detection
- Advanced ML models
- Automated network configuration recommendations
- IPv6-focused monitoring
- Distributed monitoring agents
- Cloud deployment
- Multi-user authentication
- Historical network analytics
- Network digital twin simulation
- Advanced routing simulation
- Voice-based Network Copilot
- Automated network reports

16. CONCLUSION

NetMind combines fundamental Computer Networks concepts with modern AI technologies to create an intelligent network monitoring and analysis platform. Networking remains the core of the project: packet capture, protocol analysis, performance measurement, topology modeling and routing computation provide the measured data on which every other component depends.

Machine Learning adds anomaly detection over traffic features, RAG provides knowledge grounding through a networking knowledge base, the LLM supplies natural-language reasoning and explanation, and the dashboard makes the resulting network intelligence visible. Together, these components turn raw network statistics into understandable diagnoses and recommendations while keeping the human user in control of the network. The phased plan keeps the scope realistic for a student team and provides a clear path from the core networking infrastructure to the complete integrated platform.